## Key Concepts of the Observer Pattern

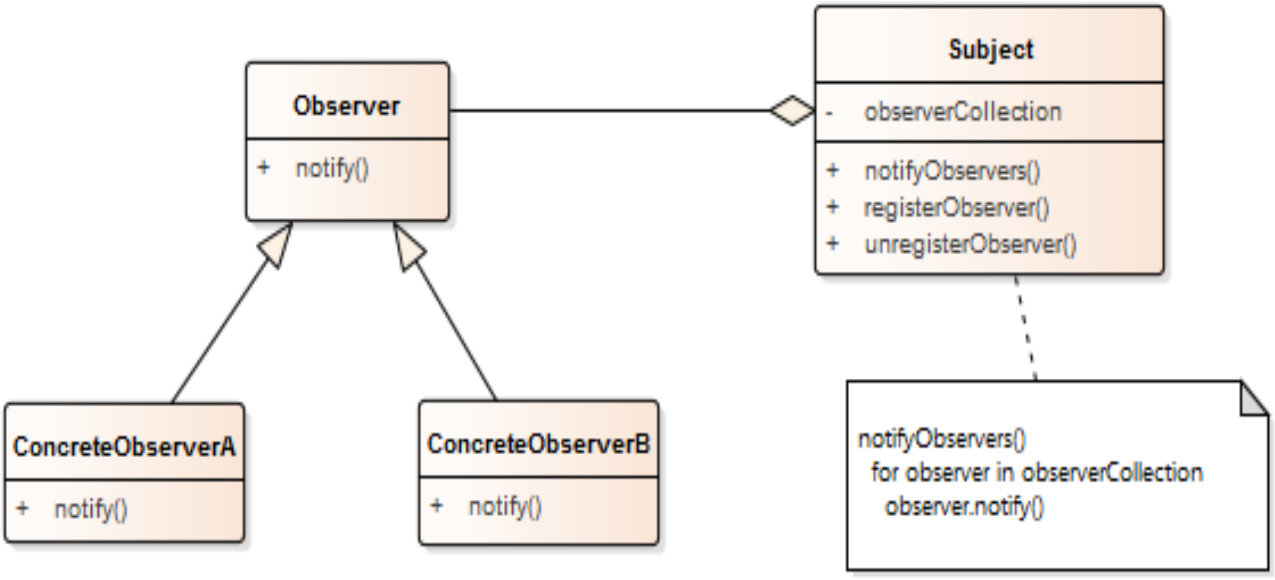
Before diving into the implementation of the Observer Pattern in C++, let's understand its key components:

- **Subject:** This is the object that is being observed. It maintains a list of observers and notifies them of state changes.

- **Observer**: Observers are objects that are interested in the state changes of the subject. They register with the subject to receive updates.

- **Concrete Subject:** The concrete subject class inherits from the subject interface or class and is responsible for maintaining the state and notifying observers when changes occur.

- **Concrete Observer:** Concrete observers implement the observer interface or inherit from an observer class. They register themselves with a concrete subject and react to state changes.



```
#include <iostream>
#include<bits/stdc++.h>
using namespace std;

//Abstract class
class Observer{
   public:
      virtual void notify() const = 0;
      virtual ~Observer(){};
};

class subject{
   list<Observer*> obj;
   public:
      subject(){
         cout<<"Sub const"<<endl;
      }
      void attach(Observer *ptr){
         cout<<typeid(*ptr).name()<<" is register"<<endl;
         obj.push_back(ptr);
      }

      void dettach(Observer *ptr){
         cout<<typeid(*ptr).name()<<" is de-register"<<endl;
         obj.remove(ptr);
      }

      void notifyObserver() const{
         for(auto ptr : obj){
            ptr->notify();
         }
      }
};

class conObserverA : public Observer{
   public:
      conObserverA(subject &sub){
         sub.attach(this);
      }
      void notify() const override{
         cout<<"conObserverA nofify"<<endl;
      }
};
```

```
class conObserverB : public Observer{
   public:
      conObserverB(subject &sub){
         sub.attach(this);
      }
      void notify() const override{
         cout<<"conObserverB nofify"<<endl;
      }
};
int main()
{
   subject sub;
   conObserverA objA(sub);
   conObserverB objB(sub);

   sub.notifyObserver();

   sub.dettach(&objA);
   sub.notifyObserver();
   return 0;
}
```